

Name: Rohit Balasaheb Kokare

Class: B.Tech (Information Technology)

Roll No.: 83

PRN No.:UIT23M1083

Assignment No:-03

Week 3 Assignment

Objective

SQL Sales Data Analysis using Subqueries, CTEs, and Window Functions

Analyze the Superstore sales dataset using SQL by applying Subqueries, Common Table Expressions (CTEs), Window Functions, and JOIN operations to solve business-related queries. The assignment focuses on identifying customer sales performance, ranking customers, and generating meaningful business insights.

Tools Used

- MySQL Workbench
- SQL
- Superstore Dataset

Dataset

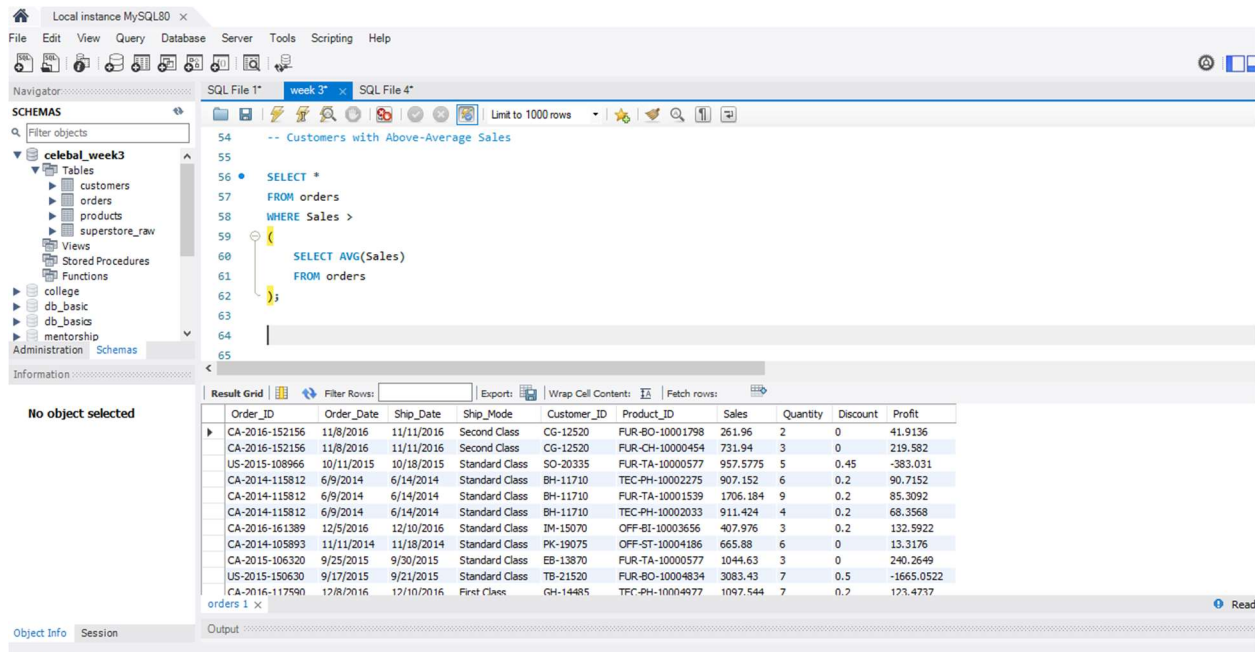
The **Superstore** dataset contains customer, order, and product information, including sales, quantity, profit, category, region, and order details. The dataset was used to create separate tables for customers, orders, and products to perform SQL analysis.

Query 1: Above Average Sales

Query:

```
SELECT *  
FROM orders  
WHERE Sales >  
(  
SELECT AVG(Sales)  
FROM orders  
);
```

Output:



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'celestial_week3' database schema with tables like customers, orders, products, and superstore_raw. The main editor window shows a SQL query: `-- Customers with Above-Average Sales`, `SELECT *`, `FROM orders`, `WHERE Sales >`, `(`, `SELECT AVG(Sales)`, `FROM orders`, `);`. The bottom panel shows the 'Result Grid' with 10 columns: Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Product_ID, Sales, Quantity, Discount, and Profit. The results list 18 orders, with the last one (CA-2016-117590) highlighted.

Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	Sales	Quantity	Discount	Profit
CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-BO-10001798	261.96	2	0	41.9136
CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0	219.582
US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.45	-383.031
CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002275	907.152	6	0.2	90.7152
CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.184	9	0.2	85.3092
CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	TEC-PH-10002033	911.424	4	0.2	68.3568
CA-2016-161389	12/5/2016	12/10/2016	Standard Class	IM-15070	OFF-BI-10003656	407.976	3	0.2	132.5922
CA-2014-105893	11/11/2014	11/18/2014	Standard Class	PK-19075	OFF-ST-10004186	665.88	6	0	13.3176
CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0	240.2649
US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.5	-1665.0522
CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	TEC-PH-10004977	1097.544	7	0.2	171.4737

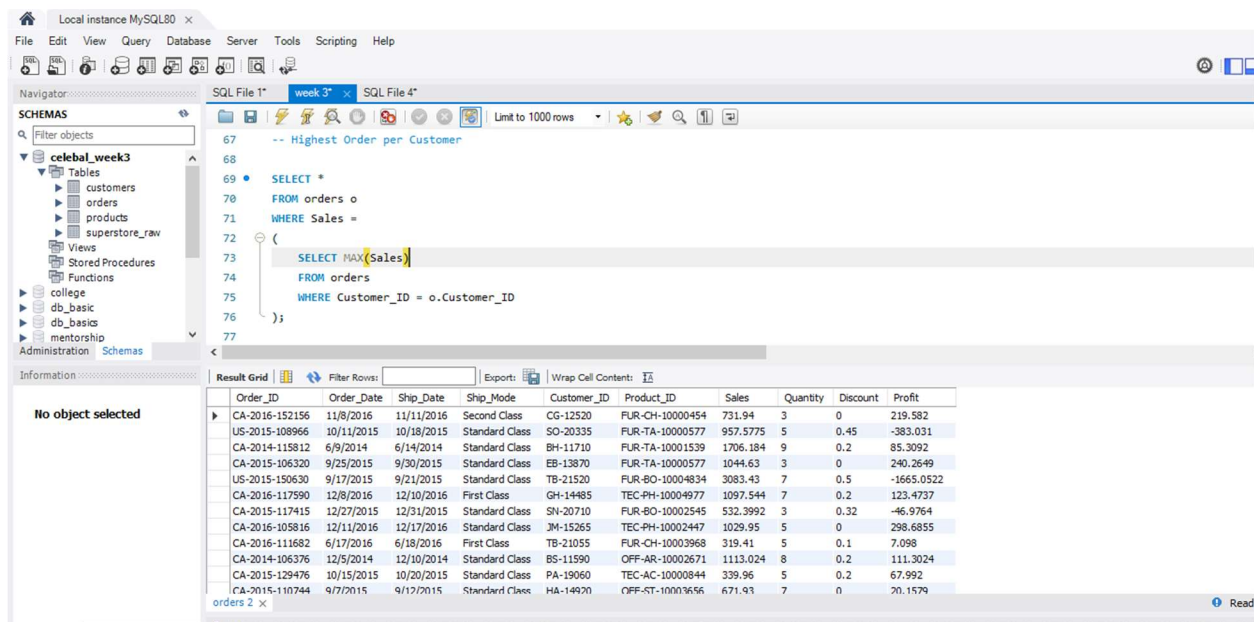
Insight: Displays orders where sales are greater than the average sales.

Query 2: Highest Order per Customer

Query:

```
SELECT *  
FROM orders o  
WHERE Sales =  
(  
    SELECT MAX(Sales)  
    FROM orders  
    WHERE Customer_ID = o.Customer_ID  
);
```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query to find the highest order per customer. The Results tab displays a table with 10 columns: Order_ID, Order_Date, Ship_Date, Ship_Mode, Customer_ID, Product_ID, Sales, Quantity, Discount, and Profit. The results are sorted by Sales in descending order.

Order_ID	Order_Date	Ship_Date	Ship_Mode	Customer_ID	Product_ID	Sales	Quantity	Discount	Profit
CA-2016-152156	11/8/2016	11/11/2016	Second Class	CG-12520	FUR-CH-10000454	731.94	3	0	219.582
US-2015-108966	10/11/2015	10/18/2015	Standard Class	SO-20335	FUR-TA-10000577	957.5775	5	0.45	-383.031
CA-2014-115812	6/9/2014	6/14/2014	Standard Class	BH-11710	FUR-TA-10001539	1706.184	9	0.2	85.3092
CA-2015-106320	9/25/2015	9/30/2015	Standard Class	EB-13870	FUR-TA-10000577	1044.63	3	0	240.2649
US-2015-150630	9/17/2015	9/21/2015	Standard Class	TB-21520	FUR-BO-10004834	3083.43	7	0.5	-1665.0522
CA-2016-117590	12/8/2016	12/10/2016	First Class	GH-14485	TEC-PH-10004977	1097.544	7	0.2	123.4737
CA-2015-117415	12/27/2015	12/31/2015	Standard Class	SN-20710	FUR-BO-10002545	532.3992	3	0.32	-46.9764
CA-2016-105816	12/11/2016	12/17/2016	Standard Class	JM-15265	TEC-PH-10002447	1029.95	5	0	298.6855
CA-2016-111682	6/17/2016	6/18/2016	First Class	TB-21055	FUR-CH-10003968	319.41	5	0.1	7.098
CA-2014-106376	12/5/2014	12/10/2014	Standard Class	BS-11590	OFF-AR-10002671	1113.024	8	0.2	111.3024
CA-2015-129476	10/15/2015	10/20/2015	Standard Class	PA-19060	TEC-AC-10000844	339.96	5	0.2	67.992
CA-2015-110744	9/7/2015	9/12/2015	Standard Class	HA-14920	OFF-ST-100019656	671.93	7	0	20.1574

Insight: Identifies the highest-value order placed by each customer.

Query 3: Total Sales per Customer (CTE)

Query:

WITH CustomerSales AS

(

SELECT Customer_ID,

SUM(Sales) AS Total_Sales

FROM orders

GROUP BY Customer_ID

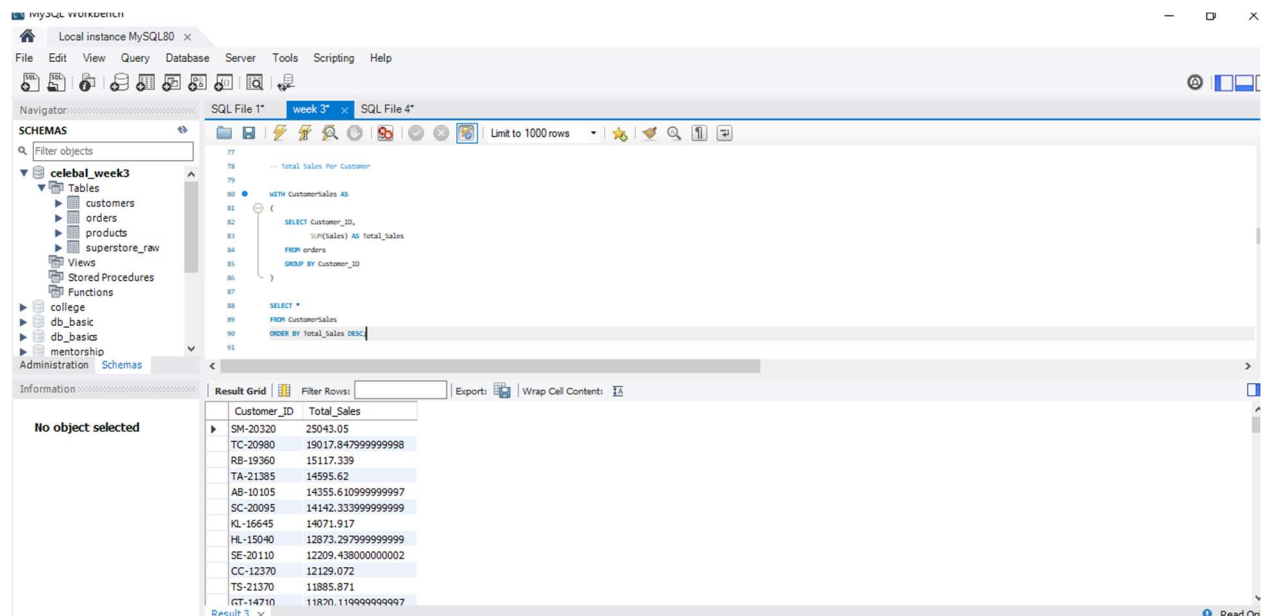
)

SELECT *

FROM CustomerSales

ORDER BY Total_Sales DESC;

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains the query for 'Total Sales per Customer' using a CTE. The 'Result Grid' tab is active, displaying the results of the query. The results are sorted by 'Total_Sales' in descending order. The first result is for Customer_ID 'SM-20320' with a 'Total_Sales' of 25043.05. The last result is for Customer_ID 'GT-14710' with a 'Total_Sales' of 11820.119999999997.

Customer_ID	Total_Sales
SM-20320	25043.05
TC-20980	19017.847999999998
RB-19360	15117.339
TA-21385	14595.62
AB-10105	14355.610999999997
SC-20095	14142.339999999999
KL-16645	14071.917
HL-15040	12873.297999999999
SE-20110	12209.438000000002
CC-12370	12129.072
TS-21370	11885.871
GT-14710	11820.119999999997

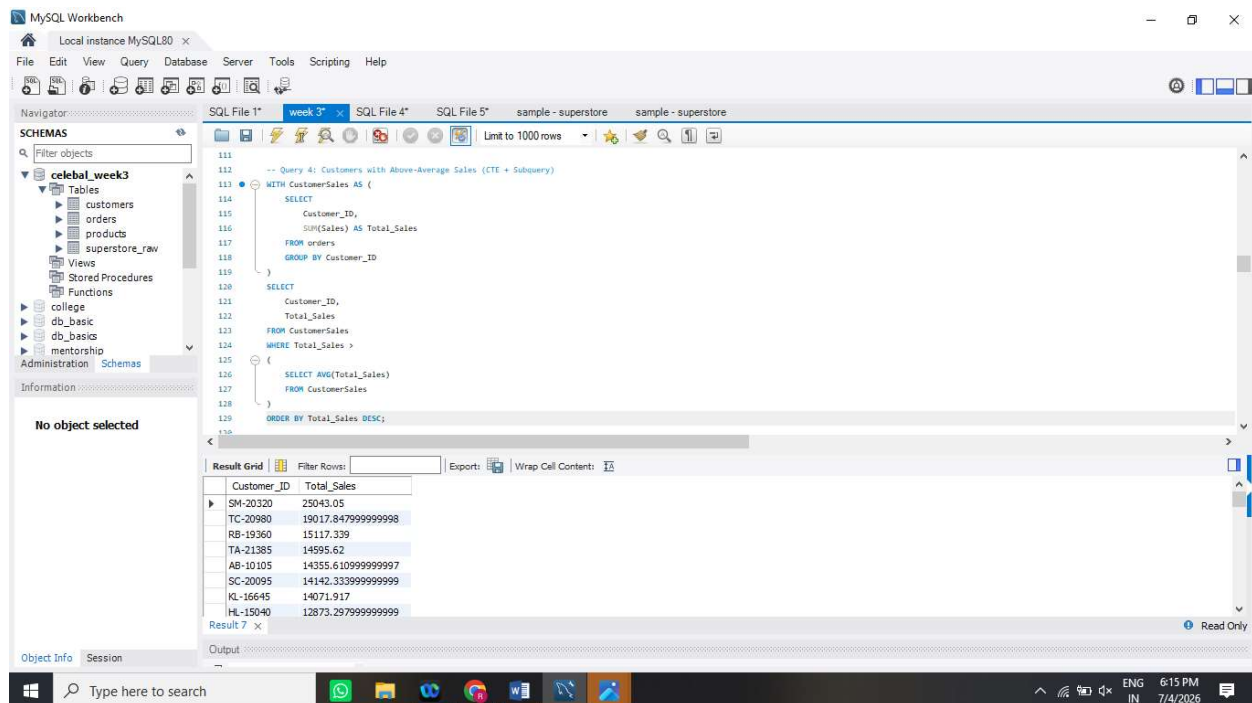
Insight: Calculates total sales for each customer.

Query 4: Customers with Above-Average Sales (CTE + Subquery)

Query:

```
WITH CustomerSales AS (  
    SELECT  
        Customer_ID,  
        SUM(Sales) AS Total_Sales  
    FROM orders  
    GROUP BY Customer_ID  
)  
SELECT  
    Customer_ID,  
    Total_Sales  
FROM CustomerSales  
WHERE Total_Sales >  
(  
    SELECT AVG(Total_Sales)  
    FROM CustomerSales  
)  
ORDER BY Total_Sales DESC;
```

Output:



Insight:

This query identifies customers whose total sales are greater than the average total sales, helping identify high-value customers.

Query 5:Assign Row Numbers to Each Order Within a Customer (Window Function + PARTITION BY)

Query:

SELECT

Customer_ID,

Order_ID,

Sales,

ROW_NUMBER() OVER (

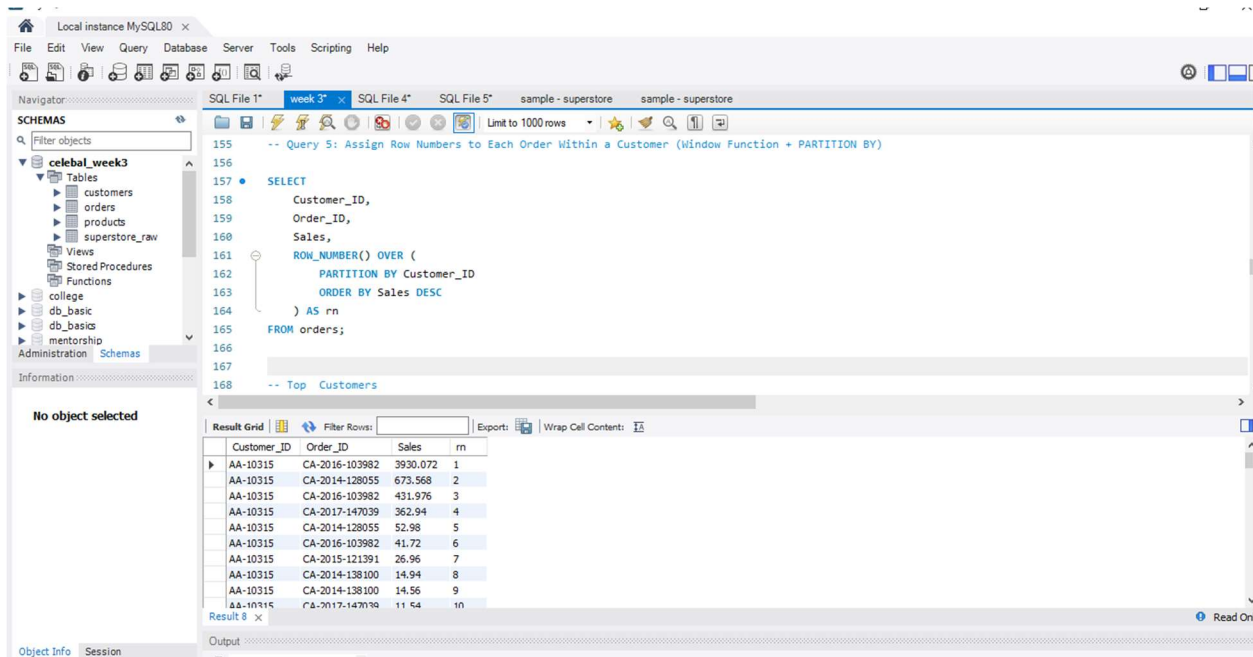
PARTITION BY Customer_ID

ORDER BY Sales DESC

) AS Row_Number

FROM orders;

Output:



The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'celestial_week3' database schema with tables 'customers', 'orders', 'products', and 'superstore_raw'. The main editor shows a SQL query for 'Query 5: Assign Row Numbers to Each Order Within a Customer (Window Function + PARTITION BY)'. The query is as follows:

```
-- Query 5: Assign Row Numbers to Each Order Within a Customer (Window Function + PARTITION BY)
SELECT
  Customer_ID,
  Order_ID,
  Sales,
  ROW_NUMBER() OVER (
    PARTITION BY Customer_ID
    ORDER BY Sales DESC
  ) AS rn
FROM orders;
```

The 'Result Grid' shows the output of the query, displaying columns 'Customer_ID', 'Order_ID', 'Sales', and 'rn'. The results are sorted by 'Sales' in descending order within each 'Customer_ID' group.

Customer_ID	Order_ID	Sales	rn
AA-10315	CA-2016-103982	3930.072	1
AA-10315	CA-2014-128055	673.568	2
AA-10315	CA-2016-103982	431.976	3
AA-10315	CA-2017-147039	362.94	4
AA-10315	CA-2014-128055	52.98	5
AA-10315	CA-2016-103982	41.72	6
AA-10315	CA-2015-121391	26.96	7
AA-10315	CA-2014-138100	14.94	8
AA-10315	CA-2014-138100	14.56	9
AA-10315	CA-2017-147039	11.64	10

Insight: This query assigns a unique row number to each order within every customer based on sales, making it easier to analyze customer-wise order rankings.

Query 6: Top 3 Customers Based on Total Sales

Query:

WITH CustomerSales AS (

SELECT

Customer_ID,

SUM(Sales) AS Total_Sales

FROM orders

GROUP BY Customer_ID

)

SELECT *

FROM (

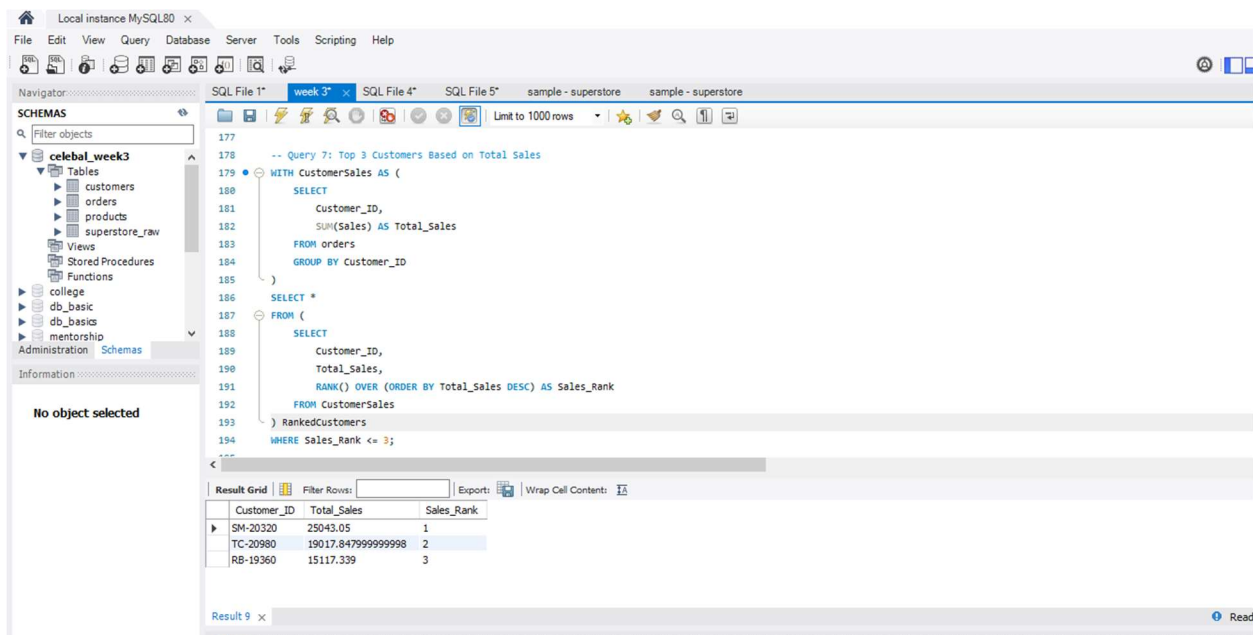
SELECT

```

Customer_ID,
Total_Sales,
RANK() OVER (ORDER BY Total_Sales DESC) AS Sales_Rank
FROM CustomerSales
) RankedCustomers
WHERE Sales_Rank <= 3;

```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query that uses a Common Table Expression (CTE) named 'RankedCustomers' to rank customers by total sales. The query is as follows:

```

-- Query 7: Top 3 Customers Based on Total Sales
WITH CustomerSales AS (
  SELECT
    Customer_ID,
    SUM(Sales) AS Total_sales
  FROM orders
  GROUP BY Customer_ID
)
SELECT *
FROM (
  SELECT
    Customer_ID,
    Total_sales,
    RANK() OVER (ORDER BY Total_sales DESC) AS Sales_Rank
  FROM CustomerSales
) RankedCustomers
WHERE Sales_Rank <= 3;

```

The results are displayed in the 'Result Grid' at the bottom, showing the top 3 customers based on total sales:

Customer_ID	Total_Sales	Sales_Rank
SM-20320	25043.05	1
TC-20980	19017.847999999998	2
RB-19360	15117.339	3

Insight: This query identifies the top three customers based on total sales using the `RANK ()` window function, helping the business recognize its highest-value customers.

Query 7 :Customer Sales Ranking Report (JOIN + CTE + Window Function)

Query:

```

WITH customer_sales AS (

```



```
SELECT
    customer_id,
    SUM(sales) AS total_sales
FROM orders
GROUP BY customer_id
)
SELECT
    c.customer_id,
    c.customer_name,
    cs.total_sales,
    RANK() OVER (ORDER BY cs.total_sales DESC) AS sales_rank
FROM customers c
JOIN customer_sales cs
ON c.customer_id = cs.customer_id
ORDER BY sales_rank;
```

Output:

The screenshot shows the MySQL Workbench interface. On the left, the 'Schemas' pane is open, showing a database named 'celebal_week3'. The main editor displays a SQL query that uses a CTE to calculate total sales per customer and then ranks them. The query is as follows:

```

-- CTE + CTE + Window Function
WITH CustomersSales AS
(
  SELECT Customer_ID,
         SUM(sales) AS Total_Sales
  FROM orders
  GROUP BY Customer_ID
)
SELECT
  c.Customer_ID,
  c.Customer_Name,
  cs.Total_Sales,
  RANK() OVER
  (
    ORDER BY cs.Total_Sales DESC
  ) AS Customer_Rank

```

The 'Result Grid' at the bottom shows the output of the query, displaying the top 5 customers by total sales:

Customer_ID	Customer_Name	Total_Sales	Customer_Rank
SM-20320	Sean Miller	25043.05	1
TC-20980	Tamara Chand	19017.847999999998	2
RB-19360	Raymond Buch	15117.339	3
TA-21385	Tom Ashbrook	14595.62	4
AB-10105	Adrian Barton	14355.610999999997	5
SC-20095	Sanjit Chand	14142.333999999999	6

Insight: This query combines customer information, total sales, and customer ranking into a single report using JOIN, CTE, and Window Functions, making customer performance analysis easier.

Mini Project: Customer Sales Insights

1. Top 5 Customers

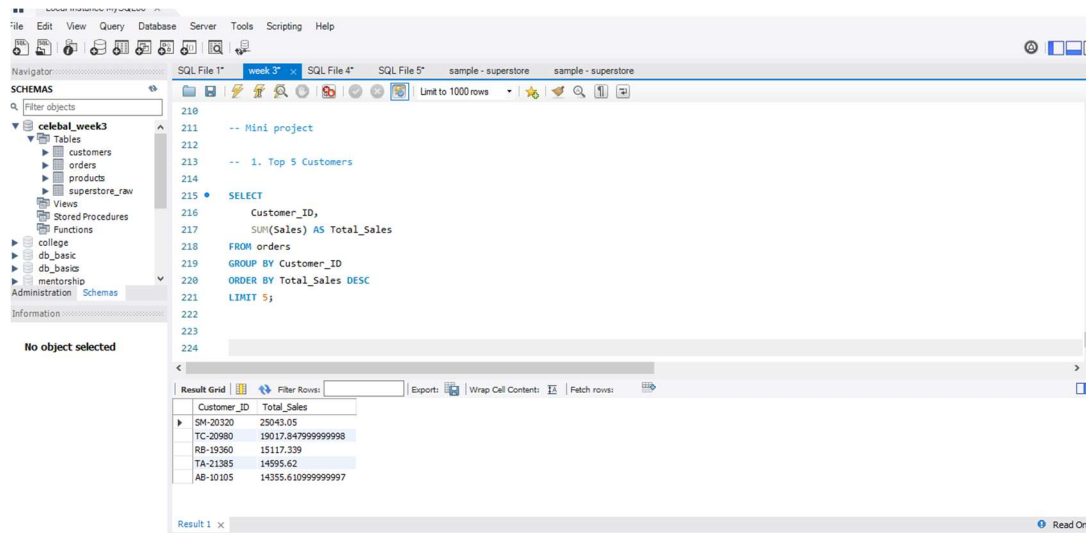
Query:

```

SELECT
  Customer_ID,
  SUM(Sales) AS Total_Sales
FROM orders
GROUP BY Customer_ID
ORDER BY Total_Sales DESC
LIMIT 5;

```

Output:



Insight: This query identifies the top five customers based on their total sales, helping the business recognize its highest-value customers.

2. Bottom 5 Customers

Query:

```
SELECT

  Customer_ID,

  SUM(Sales) AS Total_Sales

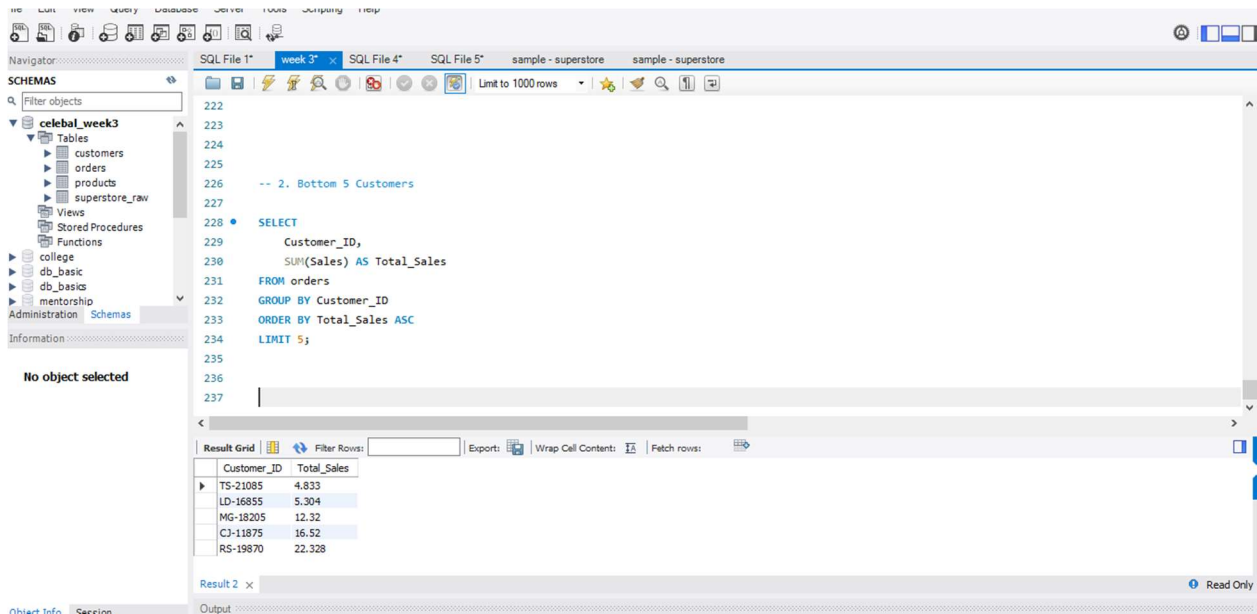
FROM orders

GROUP BY Customer_ID

ORDER BY Total_Sales ASC

LIMIT 5;
```

Output:



The screenshot shows a SQL IDE interface. On the left is a 'SCHEMAS' pane with a tree view containing 'celebal_week3' (Tables: customers, orders, products, superstore_raw; Views; Stored Procedures; Functions) and other databases like 'college', 'db_basic', 'mentorship', and 'Administration'. The main editor displays a SQL query starting with 'SELECT' and 'SUM(Sales) AS Total_Sales', followed by 'FROM orders', 'GROUP BY Customer_ID', 'ORDER BY Total_Sales ASC', and 'LIMIT 5;'. Below the editor is a 'Result Grid' showing the output of the query. The grid has two columns: 'Customer_ID' and 'Total_Sales'. The results are as follows:

Customer_ID	Total_Sales
TS-21085	4.833
LD-16855	5.304
MG-18205	12.32
CJ-11875	16.52
RS-19870	22.328

Insight: This query identifies the five customers with the lowest total sales, which helps in analyzing low-performing customer segments.

3. Customers Who Made Only One Order

Query:

SELECT

Customer_ID,

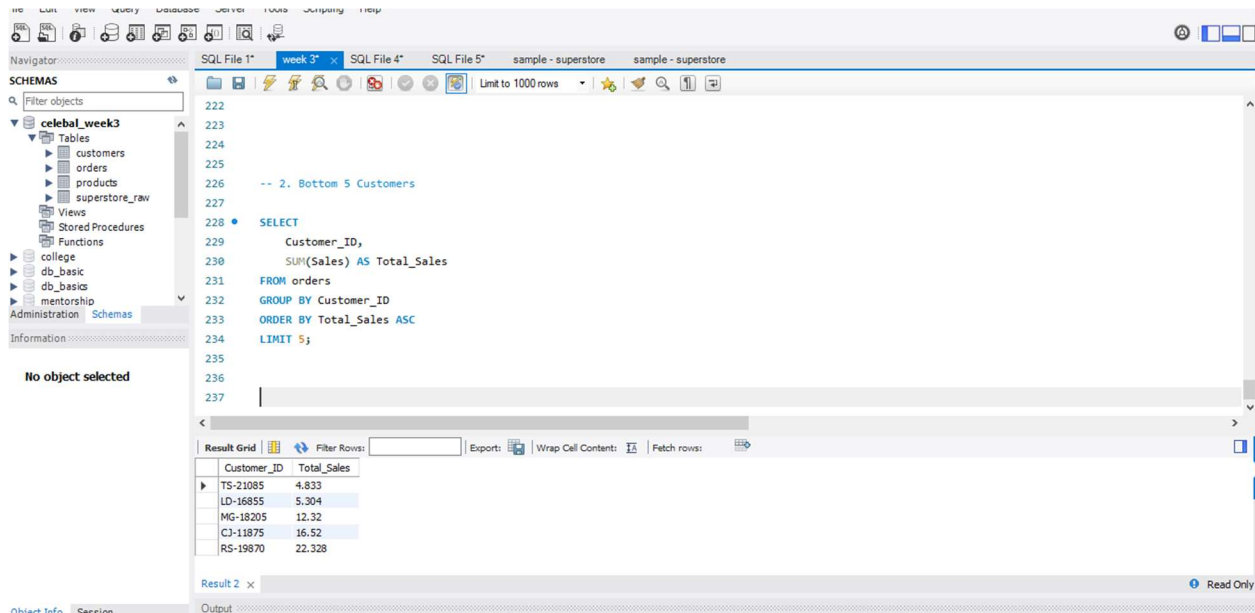
COUNT(Order_ID) AS Total_Orders

FROM orders

GROUP BY Customer_ID

HAVING COUNT(Order_ID) = 1

Output:



Insight:

This query identifies customers who placed only one order, which can help the business improve customer retention and encourage repeat purchases.

4. Customers with Above-Average Sales (CTE + Subquery)

Query:

WITH CustomerSales AS (

SELECT

Customer_ID,

SUM(Sales) AS Total_Sales

FROM orders

GROUP BY Customer_ID

```

)

SELECT

    Customer_ID,

    Total_Sales

FROM CustomerSales

WHERE Total_Sales > (

    SELECT AVG(Total_Sales)

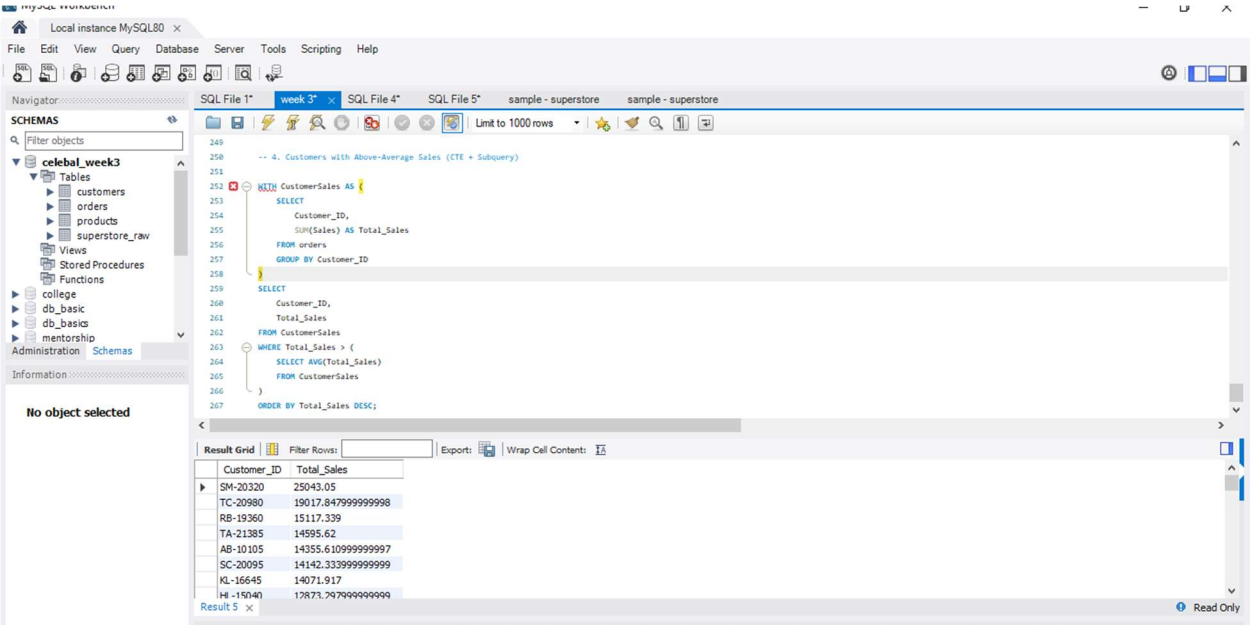
    FROM CustomerSales

)

ORDER BY Total_Sales DESC;

```

Output:



The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to find customers with above-average sales. The results grid displays the following data:

Customer_ID	Total_Sales
SM-20320	25043.05
TC-20980	19017.847999999998
RB-19360	15117.339
TA-21385	14595.62
AB-10105	14355.610999999997
SC-20095	14142.333999999999
KL-16645	14071.917
HL-15040	12873.297999999999

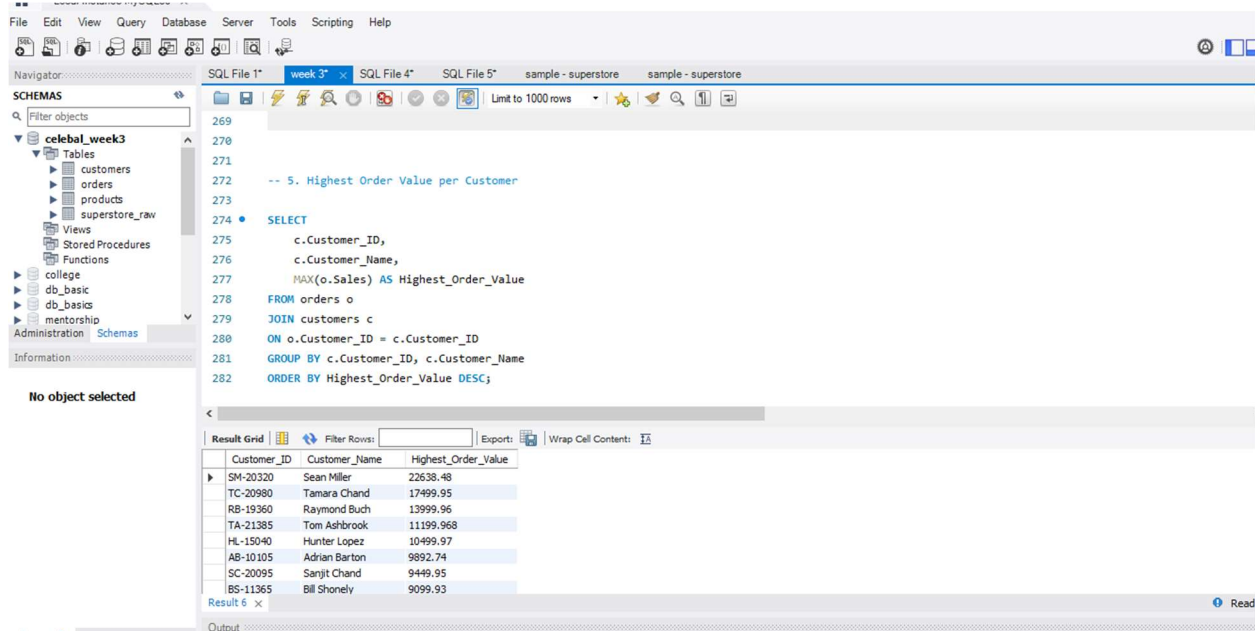
Insight: This query identifies customers whose total sales are greater than the average customer sales, highlighting high-performing customers who contribute significantly to revenue.

5. Highest Order Value per Customer

Query:

```
SELECT  
  
    c.Customer_ID,  
  
    c.Customer_Name,  
  
    MAX(o.Sales) AS Highest_Order_Value  
  
FROM orders o  
  
JOIN customers c  
  
ON o.Customer_ID = c.Customer_ID  
  
GROUP BY c.Customer_ID, c.Customer_Name  
  
ORDER BY Highest_Order_Value DESC;
```

Output:



The screenshot shows the SQL Developer interface. The left pane displays the 'celebal_week3' schema with tables 'customers', 'orders', 'products', and 'superstore_raw'. The main pane shows a SQL query (lines 269-282) that selects customer information and their highest order value. The bottom pane shows the 'Result Grid' with 6 rows of data.

```
269
270
271
272 -- 5. Highest Order Value per Customer
273
274 • SELECT
275     c.Customer_ID,
276     c.Customer_Name,
277     MAX(o.Sales) AS Highest_Order_Value
278 FROM orders o
279 JOIN customers c
280 ON o.Customer_ID = c.Customer_ID
281 GROUP BY c.Customer_ID, c.Customer_Name
282 ORDER BY Highest_Order_Value DESC;
```

Customer_ID	Customer_Name	Highest_Order_Value
SM-20320	Sean Miller	22638.48
TC-20980	Tamara Chand	17499.95
RB-19360	Raymond Buch	13999.96
TA-21385	Tom Ashbrook	11199.968
HL-15040	Hunter Lopez	10499.97
AB-10105	Adrian Barton	9892.74
SC-20095	Sanjit Chand	9449.95
BS-11365	Bill Shonely	9099.93

Insight:

This query displays the highest order value for each customer, helping identify each customer's largest purchase and spending behavior

Conclusion

- Successfully applied **Subqueries** to filter and analyze sales data.
- Used **Common Table Expressions (CTEs)** to calculate customer-wise total sales.
- Implemented **Window Functions** (RANK() and ROW_NUMBER()) to rank customers based on sales performance.
- Combined **JOIN**, **CTE**, and **Window Functions** to generate a complete customer sales report.
- Identified top-performing customers, customers with low sales, and customers with single orders, enabling meaningful business analysis.